



Image Processing and Internship

2026 CS2-23

Student ID: 240372 Sharapova Elizabet

Lab:1 Reading and Displaying Images in Python

Images Used

Three different color images were utilized throughout the experiment:

1. fb.jpg – A fruit basket image (720×720 pixels)
2. landscape1.jpg – Mountain landscape (408×612 pixels)
3. landscape2.jpg – Field landscape (360×641 pixels)



Task 1: Reading and Displaying Images

Implementation:

Installation:

```
pip install opencv-python
```

Library Import and Version Check:

```
import cv2  
print(cv2.__version__)
```

Image Loading:

```
# Read a color image using imread module of opencv library  
img = cv2.imread('fb.jpg')  
img1 = cv2.imread('landscape1.jpg')  
img2 = cv2.imread('landscape2.jpg')
```

Loaded three images (`fb.jpg`, `landscape1.jpg`, `landscape2.jpg`) using `cv2.imread()` in OpenCV's default BGR format.

Image Display:

```
# Module to display the image
cv2.imshow("Fruit Basket", img)
cv2.imshow("Landscape Mountains", img1)
cv2.imshow("Landscape Field", img2)
# Module to hold the displaying window
cv2.waitKey(0)
```

Displayed all grayscale images simultaneously using `cv2.imshow()`. Kept windows open with `cv2.waitKey(0)`

Dimension Extraction:

```
print(img.shape)
print(img1.shape)
print(img2.shape)

(720, 720, 3)
(408, 612, 3)
(360, 641, 3)
```

Checked image dimensions using the `.shape` attribute and printed them as (height, width, channels): `img` → (720, 720, 3), `img1` → (408, 612, 3), `img2` → (360, 641, 3).

Task 2: Converting Images to Grayscale

A. Method: Direct Grayscale Loading (without `cvtColor`)

Method A: Converting without `cvtColor` (Direct Read)

```
# Read and convert the image to grayscale directly
gray_direct = cv2.imread('fb.jpg', cv2.IMREAD_GRAYSCALE)
gray_direct1 = cv2.imread('landscape1.jpg', cv2.IMREAD_GRAYSCALE)
gray_direct2 = cv2.imread('landscape2.jpg', cv2.IMREAD_GRAYSCALE)
```

B. Method: Color-to-Grayscale Conversion (with `cvtColor`)

Method B: Converting with `cvtColor` module

```
# Convert from BGR to Grayscale using cvtColor
gray_cvt = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
gray_cvt1 = cv2.cvtColor(img1, cv2.COLOR_BGR2GRAY)
gray_cvt2 = cv2.cvtColor(img2, cv2.COLOR_BGR2GRAY)
```

Displayed all grayscale images simultaneously using `cv2.imshow()`. Kept windows open with `cv2.waitKey(0)`

```
# Module to display the image
cv2.imshow('Grayscale - Direct Read', gray_direct)
cv2.imshow('Grayscale - Direct Read1', gray_direct1)
cv2.imshow('Grayscale - Direct Read2', gray_direct2)
# Module to hold the displaying window
cv2.waitKey(0)
```

```
# Module to display the image
cv2.imshow('Grayscale - Converted', gray_cvt)
cv2.imshow('Grayscale - Converted1', gray_cvt1)
cv2.imshow('Grayscale - Converted2', gray_cvt2)
# Module to hold the displaying window
cv2.waitKey(0)
```

Used `cv2.imwrite()` function to save processed images

```
# Save the result
cv2.imwrite('gray_direct.jpg', gray_direct)
cv2.imwrite('gray_direct1.jpg', gray_direct1)
cv2.imwrite('gray_direct2.jpg', gray_direct2)
```

```
# Save the result
cv2.imwrite('gray_cvt.jpg', gray_cvt)
cv2.imwrite('gray_cvt1.jpg', gray_cvt1)
cv2.imwrite('gray_cvt2.jpg', gray_cvt2)
```

Results for Task 2

Direct Grayscale Loading (without cvtColor):



B. Method: Color-to-Grayscale Conversion (with cvtColor):



Task 3: Thresholding Experiments

I performed three distinct thresholding experiments on three different images, creating a total of 9 binary images.

For EACH threshold value (100, 128, 180):

1. Applied `cv2.threshold()` function to convert grayscale to binary
2. Used `cv2.THRESH_BINARY` method for standard binary thresholding
3. Saved results after processing

Task 3: Thresholding Experiments 1 & 2. Experiment with different threshold values and Save results

Step 3: Thresholding to Create a Binary Image

```
# --- Experiment 1: Low Threshold (100) ---
_, binary_100 = cv2.threshold(gray_cvt, 100, 255, cv2.THRESH_BINARY)
_, binary1_100 = cv2.threshold(gray_cvt1, 100, 255, cv2.THRESH_BINARY)
_, binary2_100 = cv2.threshold(gray_cvt2, 100, 255, cv2.THRESH_BINARY)
cv2.imwrite('binary_100.jpg', binary_100) # Save image
cv2.imwrite('binary_100.1.jpg', binary1_100)
cv2.imwrite('binary_100.2.jpg', binary2_100)
```

True

```
# --- Experiment 2: Mid Threshold (128) ---
_, binary_128 = cv2.threshold(gray_cvt, 128, 255, cv2.THRESH_BINARY)
_, binary1_128 = cv2.threshold(gray_cvt1, 128, 255, cv2.THRESH_BINARY)
_, binary2_128 = cv2.threshold(gray_cvt2, 128, 255, cv2.THRESH_BINARY)
cv2.imwrite('binary_128.jpg', binary_128) # Save image
cv2.imwrite('binary_128.1.jpg', binary1_128)
cv2.imwrite('binary_128.2.jpg', binary2_128)
```

True

```
# --- Experiment 3: High Threshold (180) ---
_, binary_180 = cv2.threshold(gray_cvt, 180, 255, cv2.THRESH_BINARY)
_, binary1_180 = cv2.threshold(gray_cvt1, 180, 255, cv2.THRESH_BINARY)
_, binary2_180 = cv2.threshold(gray_cvt2, 180, 255, cv2.THRESH_BINARY)
cv2.imwrite('binary_180.jpg', binary_180) # Save image
cv2.imwrite('binary_180.1.jpg', binary1_180)
cv2.imwrite('binary_180.2.jpg', binary2_180)
```

True

```
# Saving the binary images
cv2.imwrite('binary_100.jpg', binary_100)
cv2.imwrite('binary_100.1.jpg', binary1_100)
cv2.imwrite('binary_100.2.jpg', binary2_100)
cv2.imwrite('binary_128.jpg', binary_128)
cv2.imwrite('binary_128.1.jpg', binary1_128)
cv2.imwrite('binary_128.2.jpg', binary2_128)
cv2.imwrite('binary_180.jpg', binary_180)
cv2.imwrite('binary_180.1.jpg', binary1_180)
cv2.imwrite('binary_180.2.jpg', binary2_180)
```

True

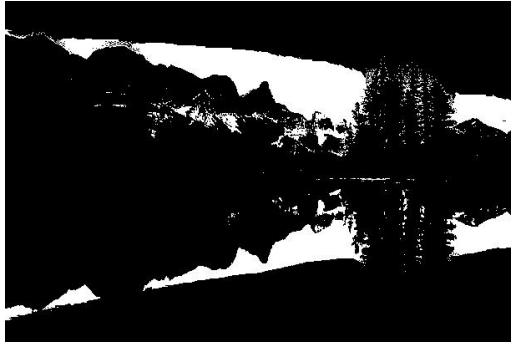
Results for threshold 100:



Results for threshold 128:



Results for threshold 180:



Conclusion:

Through these tasks, I gained practical experience in image processing with OpenCV, including loading, displaying, and converting images between color spaces, applying thresholds, and comparing results across multiple images. I learned to choose appropriate methods, tune parameters, assess image quality, and manage files systematically. Overall, the tasks reinforced fundamental concepts and prepared me for more advanced computer vision techniques.